

Федеральное государственное автономное образовательное учреждение высшего образования
«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Лабораторная работа №2
по курсу «Вычислительная математика»
Вариант 19

Выполнил:

Якименко Владислав Игоревич, Р3213

Проверила:

Малышева Татьяна Алексеевна

Содержание

1	Вычислительная реализация задачи	1
1.1	Часть. Решение нелинейного уравнения	1
1.1.1	Отделение корней (графический метод)	1
1.1.2	Интервалы изоляции корней	2
1.1.3	Уточнение корней ($\text{eps} = 10^{-2}$)	2
1.2	Часть. Решение системы нелинейных уравнений	4
1.2.1	Отделение корней (графический метод)	4
1.2.2	Приведение к виду $x = \text{phi1}(x,y)$, $y = \text{phi2}(x,y)$	5
1.2.3	Проверка условия сходимости	5
1.2.4	Уточнение решения ($\text{eps} = 10^{-2}$)	5
2	Программная реализация	6
2.1	Блок-схемы алгоритмов	6
2.1.1	Блок-схема метода хорд	6
2.1.2	Блок-схема метода Ньютона	7
2.1.3	Блок-схема метода простой итерации	8
2.1.4	Блок-схема метода Ньютона для системы	9
2.2	Листинги программ	9
2.3	Скриншоты работы web-интерфейса	13
3	Вывод	16

1 Вычислительная реализация задачи

1.1 Часть. Решение нелинейного уравнения

Задание:

1. Отделить корни заданного нелинейного уравнения графически.
2. Определить интервалы изоляции корней.
3. Уточнить корни нелинейного уравнения с точностью $\varepsilon = 10^{-2}$.
4. Вычисления оформить в виде таблиц.

Уравнение:

$$f(x) = 2x^3 - 1.89x^2 - 5x + 2.34$$

Производная:

$$f'(x) = 6x^2 - 3.78x - 5$$

Вторая производная:

$$f''(x) = 12x - 3.78$$

1.1.1 Отделение корней (графический метод)

Построим график функции $f(x)$ на интервале $[-3; 3]$.

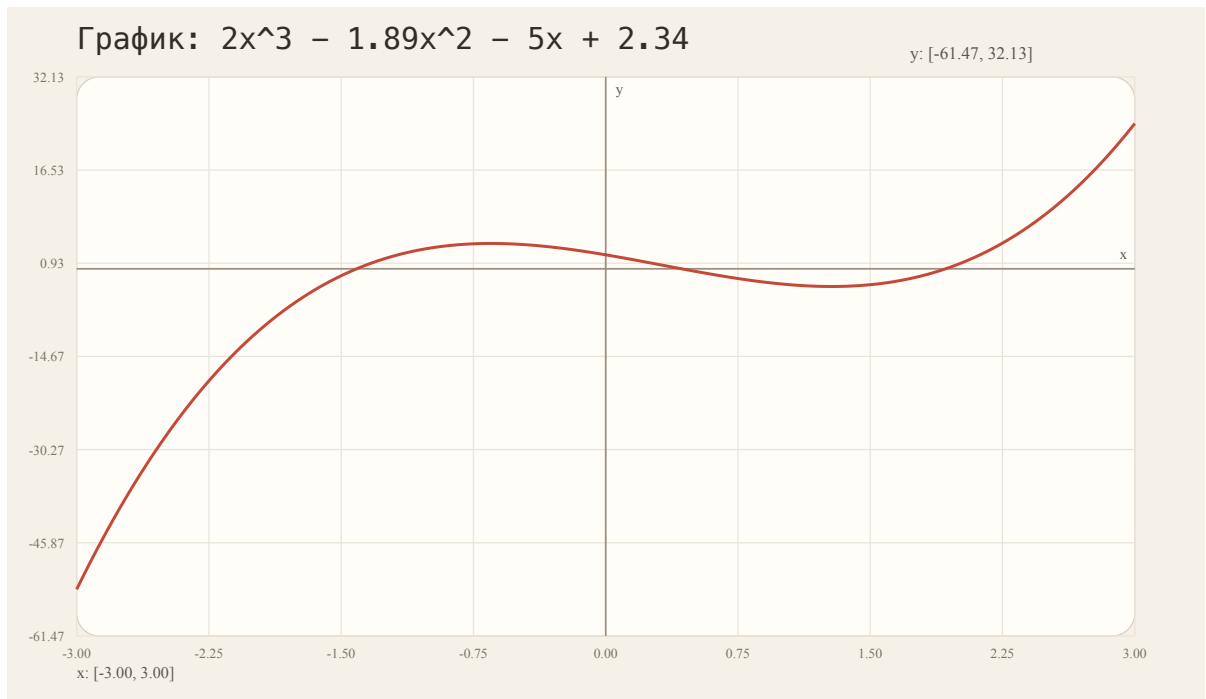


Рис. 1: График функции $f(x) = 2x^3 - 1.89x^2 - 5x + 2.34$

По графику видно, что уравнение имеет три действительных корня:

$$x_1 \approx -1.41, \quad x_2 \approx 0.43, \quad x_3 \approx 1.93.$$

1.1.2 Интервалы изоляции корней

Для левого корня выберем интервал $[-1.50; -1.25]$:

$$f(-1.50) = -1.162 < 0, \quad f(-1.25) = 1.731 > 0.$$

Для центрального корня выберем интервал $[0.00; 1.00]$:

$$f(0.00) = 2.340 > 0, \quad f(1.00) = -2.550 < 0.$$

Для правого корня выберем интервал $[1.50; 2.00]$:

$$f(1.50) = -2.475 < 0, \quad f(2.00) = 0.780 > 0.$$

Таким образом, на каждом из выбранных интервалов происходит смена знака функции, значит корень на интервале существует.

1.1.3 Уточнение корней ($\varepsilon = 10^{-2}$)

В вычислительной части использованы методы, которые уже подготовлены в проекте для отчёта:

Корень	Интервал изоляции	Метод
Левый x_1	$[-1.50; -1.25]$	половинного деления
Центральный x_2	$[0.00; 1.00]$	секущих
Правый x_3	$[1.50; 2.00]$	простой итерации

Левый корень. Метод половинного деления. Рабочая формула:

$$x = \frac{a + b}{2}.$$

Критерий остановки:

$$|b - a| \leq \varepsilon \quad \text{или} \quad |f(x)| \leq \varepsilon.$$

Получено:

$$x_1 \approx -1.410156, \quad f(x_1) \approx 0.024133, \quad n = 6.$$

Таблица 1: Метод половинного деления для левого корня

№	a	b	x	$f(a)$	$f(b)$	$f(x)$	$ b - a $
1	-1.500	-1.250	-1.375	-1.162	1.731	0.442	0.250
2	-1.500	-1.375	-1.438	-1.162	0.442	-0.319	0.125
3	-1.438	-1.375	-1.406	-0.319	0.442	0.072	0.063
4	-1.438	-1.406	-1.422	-0.319	0.072	-0.121	0.031
5	-1.422	-1.406	-1.414	-0.121	0.072	-0.024	0.016
6	-1.414	-1.406	-1.410	-0.024	0.072	0.024	0.008

Центральный корень. Метод секущих. Рабочая формула:

$$x_{k+1} = x_k - \frac{f(x_k)(x_k - x_{k-1})}{f(x_k) - f(x_{k-1})}.$$

Критерий остановки:

$$|x_{k+1} - x_k| \leq \varepsilon.$$

Получено:

$$x_2 \approx 0.430001, \quad f(x_2) \approx -0.000454, \quad n = 3.$$

Таблица 2: Метод секущих для центрального корня

№	x_{i-1}	x_i	x_{i+1}	$f(x_{i+1})$	$ x_{i+1} - x_i $
1	0.000	1.000	0.479	-0.266	0.521
2	1.000	0.479	0.418	0.067	0.061
3	0.479	0.418	0.430	-0.000	0.012

Правый корень. Метод простой итерации. Использована итерационная форма

$$x_{k+1} = \varphi(x_k) = x_k + \lambda f(x_k),$$

где коэффициент сжатия

$$q = \max |\varphi'(x)| = 0.752622 < 1.$$

Получено:

$$x_3 \approx 1.927069, \quad f(x_3) \approx -0.001336, \quad n = 4.$$

Таблица 3: Метод простой итерации для правого корня

№	x_i	x_{i+1}	$\varphi(x_{i+1})$	$ x_{i+1} - x_i $	Оценка погрешности
1	1.750	1.879	1.919	0.129	0.393
2	1.879	1.919	1.926	0.040	0.122
3	1.919	1.926	1.927	0.007	0.021
4	1.926	1.927	1.927	0.001	0.003

Итог по уравнению.

Корень	Метод	x^*	$f(x^*)$	Итераций
Левый	половинного деления	-1.410	0.024	6
Центральный	секущих	0.430	-0.000	3
Правый	простой итерации	1.927	-0.001	4

1.2 Часть. Решение системы нелинейных уравнений

Задание:

1. Отделить корни заданной системы нелинейных уравнений графически.
2. Привести систему к виду, пригодному для итерационного процесса.
3. Проверить условие сходимости итерационного процесса.
4. Уточнить решение системы с точностью $\varepsilon = 10^{-2}$.
5. Вычисления оформить в виде таблиц.

Система:

$$\begin{cases} \sin(y - 1) + x = 1.3, \\ y - \sin(x + 1) = 0.8. \end{cases}$$

Метод: простая итерация.

1.2.1 Отделение корней (графический метод)

Построим график кривых $F_1(x, y) = 0$ и $F_2(x, y) = 0$ на области, содержащей точку пересечения.

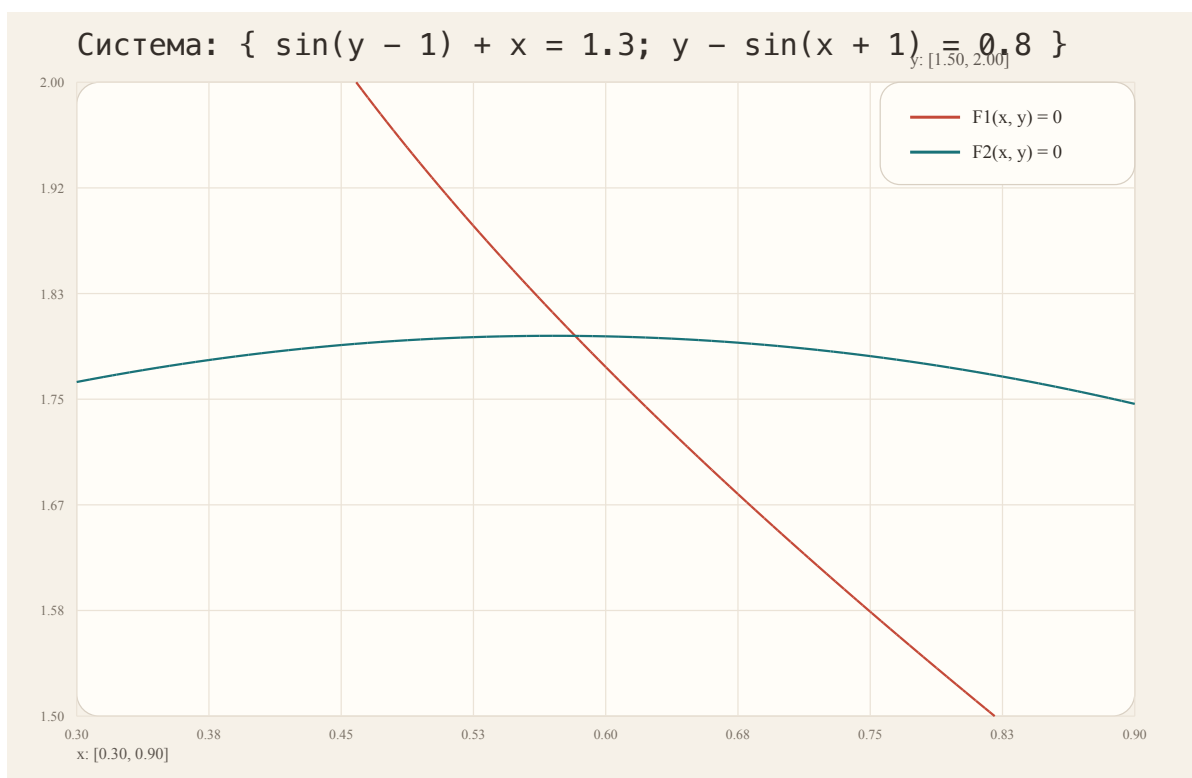


Рис. 2: Графическое отделение корня системы варианта 19

По графику видно, что точка пересечения находится вблизи

$$(x_0, y_0) \approx (0.58, 1.80).$$

1.2.2 Приведение к виду $x = \varphi_1(x, y)$, $y = \varphi_2(x, y)$

Для метода простой итерации систему запишем в виде

$$\begin{cases} x = 1.3 - \sin(y - 1), \\ y = 0.8 + \sin(x + 1). \end{cases}$$

Тогда

$$\Phi(x, y) = \begin{pmatrix} 1.3 - \sin(y - 1) \\ 0.8 + \sin(x + 1) \end{pmatrix}.$$

1.2.3 Проверка условия сходимости

Матрица Якоби отображения $\Phi(x, y)$ имеет вид:

$$D\Phi(x, y) = \begin{pmatrix} 0 & -\cos(y - 1) \\ \cos(x + 1) & 0 \end{pmatrix}.$$

На прямоугольнике

$$[0.50; 0.65] \times [1.75; 1.85]$$

получаем

$$q = \max \|D\Phi(x, y)\|_{\infty} \approx 0.731689 < 1.$$

Следовательно, достаточное условие сходимости метода простой итерации выполняется.

1.2.4 Уточнение решения ($\varepsilon = 10^{-2}$)

Рабочие формулы:

$$\begin{cases} x_{k+1} = 1.3 - \sin(y_k - 1), \\ y_{k+1} = 0.8 + \sin(x_k + 1). \end{cases}$$

Получено:

$$x^* \approx 0.583432, \quad y^* \approx 1.799908.$$

Невязки:

$$F_1(x^*, y^*) \approx 0.000724, \quad F_2(x^*, y^*) \approx -0.000013.$$

Таблица 4: Метод простой итерации для системы варианта 19

№	x_i	y_i	x_{i+1}	y_{i+1}	max diff	Оценка погрешности
1	0.500	1.750	0.618	1.797	0.118	0.323
2	0.618	1.797	0.584	1.799	0.034	0.093
3	0.584	1.799	0.583	1.800	0.001	0.003

Итог по системе.

$$(x^*, y^*) \approx (0.583, 1.800), \quad n = 3.$$

2 Программная реализация

В программной части варианта 19 реализованы методы 2, 3, 5 и 6 из задания:

- метод хорд;
- метод Ньютона;
- метод простой итерации для одного уравнения;
- метод Ньютона для системы нелинейных уравнений.

Проект написан на JavaScript. Консольная часть запускается через Node.js, а для визуализации дополнительно сделан GUI в браузере. Программа умеет принимать входные данные с клавиатуры или из JSON-файла, проверяет корректность интервала изоляции, строит графики и выводит историю итераций.

2.1 Блок-схемы алгоритмов

2.1.1 Блок-схема метода хорд

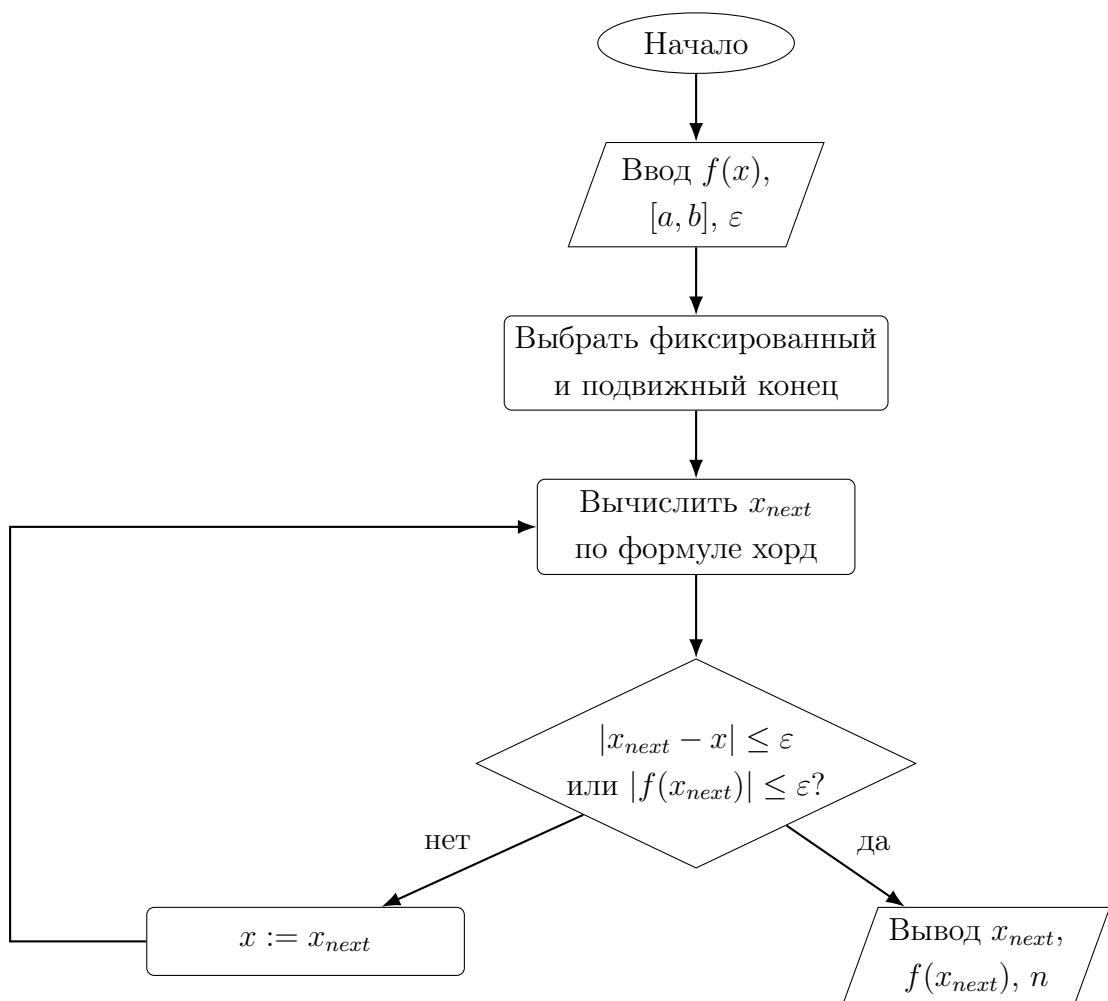


Рис. 3: Блок-схема метода хорд с неподвижным концом

2.1.2 Блок-схема метода Ньютона

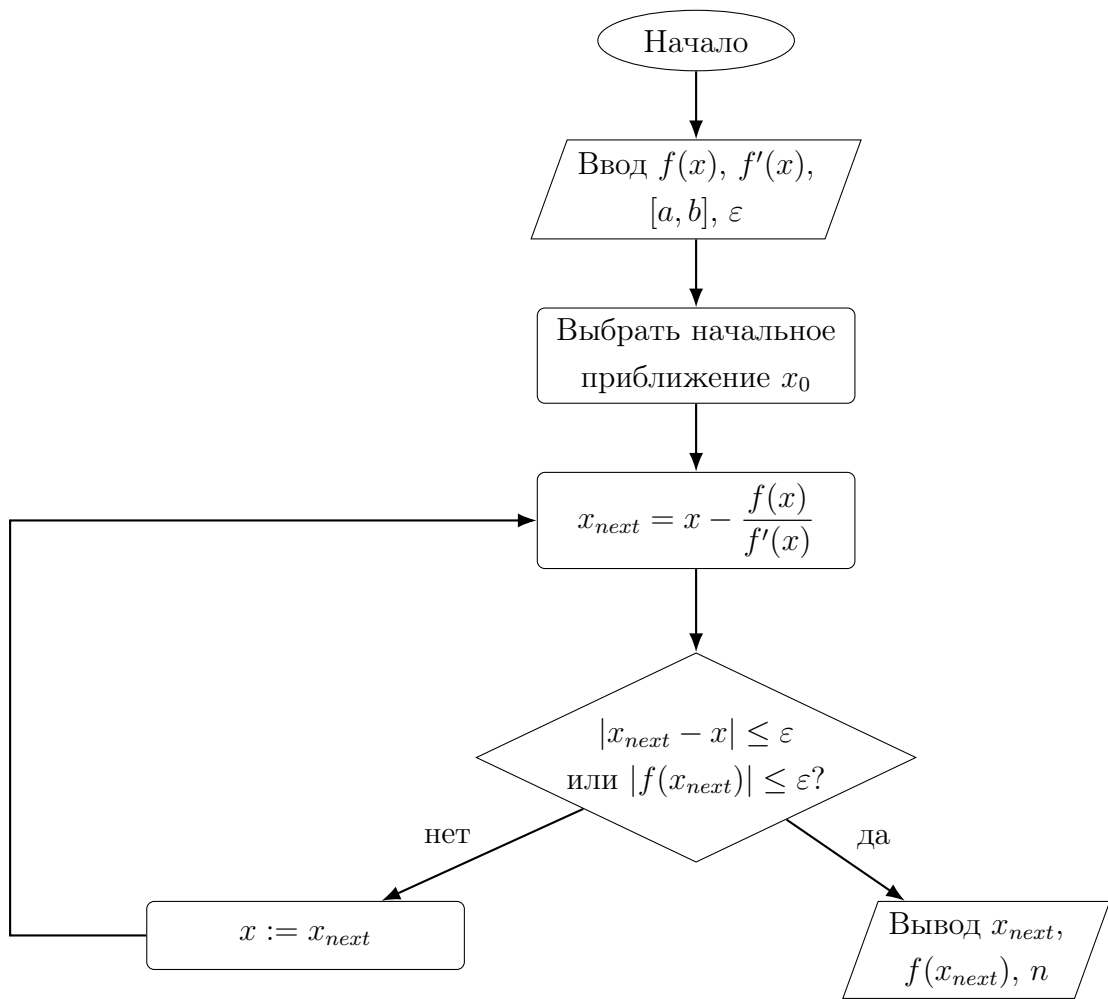


Рис. 4: Блок-схема метода Ньютона

2.1.3 Блок-схема метода простой итерации

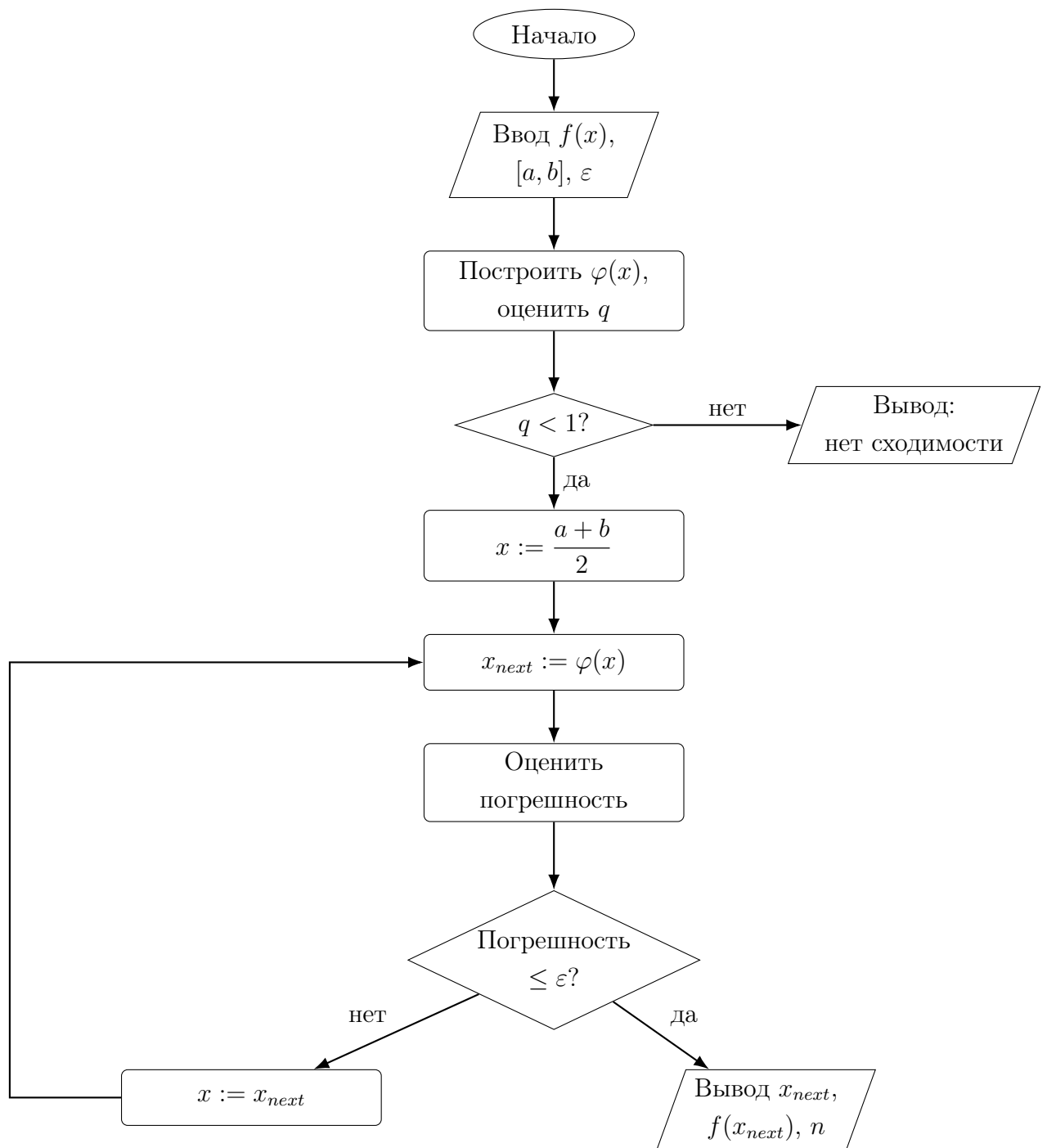


Рис. 5: Блок-схема метода простой итерации

2.1.4 Блок-схема метода Ньютона для системы

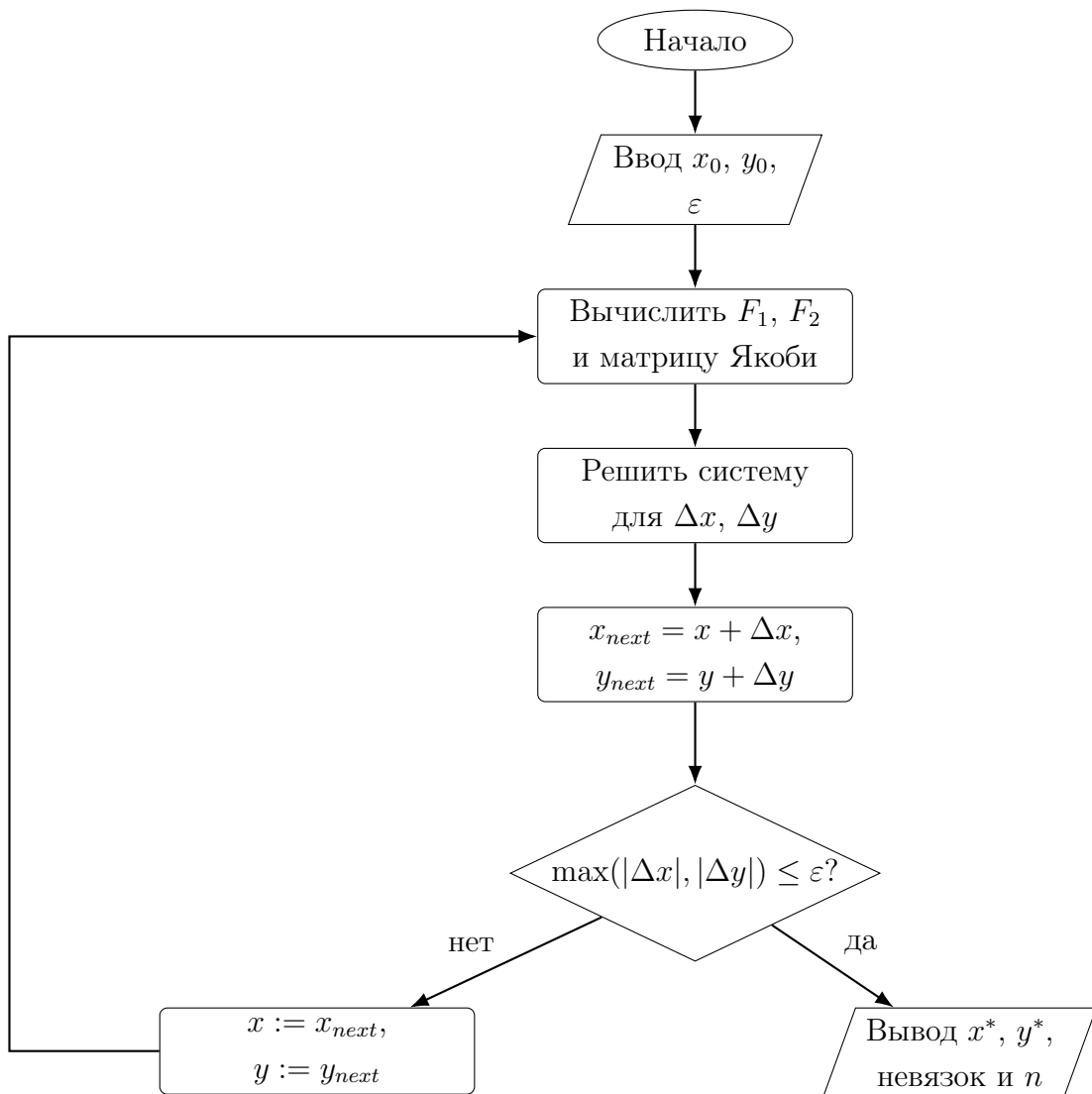


Рис. 6: Блок-схема метода Ньютона для системы

2.2 Листинги программ

Язык реализации: JavaScript (Node.js). Для вычислительных методов не используются сторонние библиотеки. Ниже приведены ключевые фрагменты из основных файлов с реализациями:

- src/methods/equations.js;
- src/methods/systems.js.

Листинг 1: Метод хорд с неподвижным концом (фрагмент src/methods/equations.js)

```
1 export function chordFixed(equation, a, b, epsilon, maxIterations = 1000) {
2   const history = [];
3   const { fixed, moving } = chooseChordConfiguration(equation, a, b);
4   const fFixed = equation.f(fixed);
```

```

5   let current = moving;
6
7   for (let iteration = 1; iteration <= maxIterations; iteration += 1) {
8       const fCurrent = equation.f(current);
9       const denominator = fFixed - fCurrent;
10      if (Math.abs(denominator) < 1e-14) {
11          throw new Error("...");
12      }
13
14      const next = current - (fCurrent * (fixed - current)) / denominator;
15      const fNext = equation.f(next);
16      const diff = Math.abs(next - current);
17
18      history.push({
19          iteration,
20          fixed,
21          moving: current,
22          next,
23          fFixed,
24          fMoving: fCurrent,
25          fNext,
26          diff,
27      });
28
29      if (diff <= epsilon || Math.abs(fNext) <= epsilon) {
30          return { method: "chord", root: next, fx: fNext, iterations: iteration,
31              history };
32      }
33      current = next;
34  }
35  throw new Error("...");

```

ЛИСТИНГ 2: Метод Ньютона (фрагмент src/methods/equations.js)

```

1   export function newton(equation, a, b, epsilon, maxIterations = 1000) {
2       const history = [];
3       let current = chooseInitialByCurvature(equation, a, b);
4
5       for (let iteration = 1; iteration <= maxIterations; iteration += 1) {
6           const fx = equation.f(current);
7           const dfx = equation.df(current);
8
9           if (Math.abs(dfx) < 1e-14) {
10              throw new Error("...");

```

```

11     }
12
13     const next = current - fx / dfx;
14     const diff = Math.abs(next - current);
15     const fNext = equation.f(next);
16
17     history.push({ iteration, x: current, fx, dfx, next, diff, fNext });
18
19     if (diff <= epsilon || Math.abs(fNext) <= epsilon) {
20         return { method: "newton", root: next, fx: fNext, iterations: iteration,
21             history };
22     }
23     current = next;
24 }
25 throw new Error("...");
26 }

```

Листинг 3: Метод простой итерации (фрагмент src/methods/equations.js)

```

1 export function simpleIterationEquation(equation, a, b, epsilon, maxIterations =
2     1000) {
3     const { min, max, maxAbs } = estimateDerivativeStats(equation.df, a, b);
4     const midpoint = (a + b) / 2;
5     const signProbe = Math.sign(equation.df(midpoint)) || Math.sign(min + max) || 1;
6     const lambda = -signProbe / maxAbs;
7     const phi = (x) => x + lambda * equation.f(x);
8     const q = estimateMaxAbs((x) => 1 + lambda * equation.df(x), a, b);
9
10    if (q >= 1) {
11        throw new Error("...");
12    }
13
14    let current = midpoint;
15    for (let iteration = 1; iteration <= maxIterations; iteration += 1) {
16        const next = phi(current);
17        const diff = Math.abs(next - current);
18        const estimatedError = q === 0 ? diff : (q / (1 - q)) * diff;
19
20        if (estimatedError <= epsilon || Math.abs(equation.f(next)) <= epsilon) {
21            return { method: "simpleIteration", root: next, fx: equation.f(next),
22                iterations: iteration };
23        }
24        current = next;
25    }
26    throw new Error("...");
27 }

```

25 }

Листинг 4: Метод Ньютона для системы (фрагмент `src/methods/systems.js`)

```
1 export function newtonSystem(system, initial, epsilon, maxIterations = 100) {
2   const history = [];
3   let current = { ...initial };
4
5   for (let iteration = 1; iteration <= maxIterations; iteration += 1) {
6     const [f1, f2] = system.f(current.x, current.y);
7     const jacobian = system.jacobian(current.x, current.y);
8     const [dx, dy] = solveLinear2x2(jacobian, [-f1, -f2]);
9
10    const next = {
11      x: current.x + dx,
12      y: current.y + dy,
13    };
14    const [nextF1, nextF2] = system.f(next.x, next.y);
15    const errors = [Math.abs(dx), Math.abs(dy)];
16
17    history.push({
18      iteration,
19      x: current.x,
20      y: current.y,
21      f1,
22      f2,
23      dx,
24      dy,
25      nextX: next.x,
26      nextY: next.y,
27      nextF1,
28      nextF2,
29    });
30
31    if (maxAbs(errors) <= epsilon) {
32      return { method: "newtonSystem", solution: next, residuals: [nextF1, nextF2], iterations: iteration };
33    }
34    current = next;
35  }
36  throw new Error("...");
37 }
```

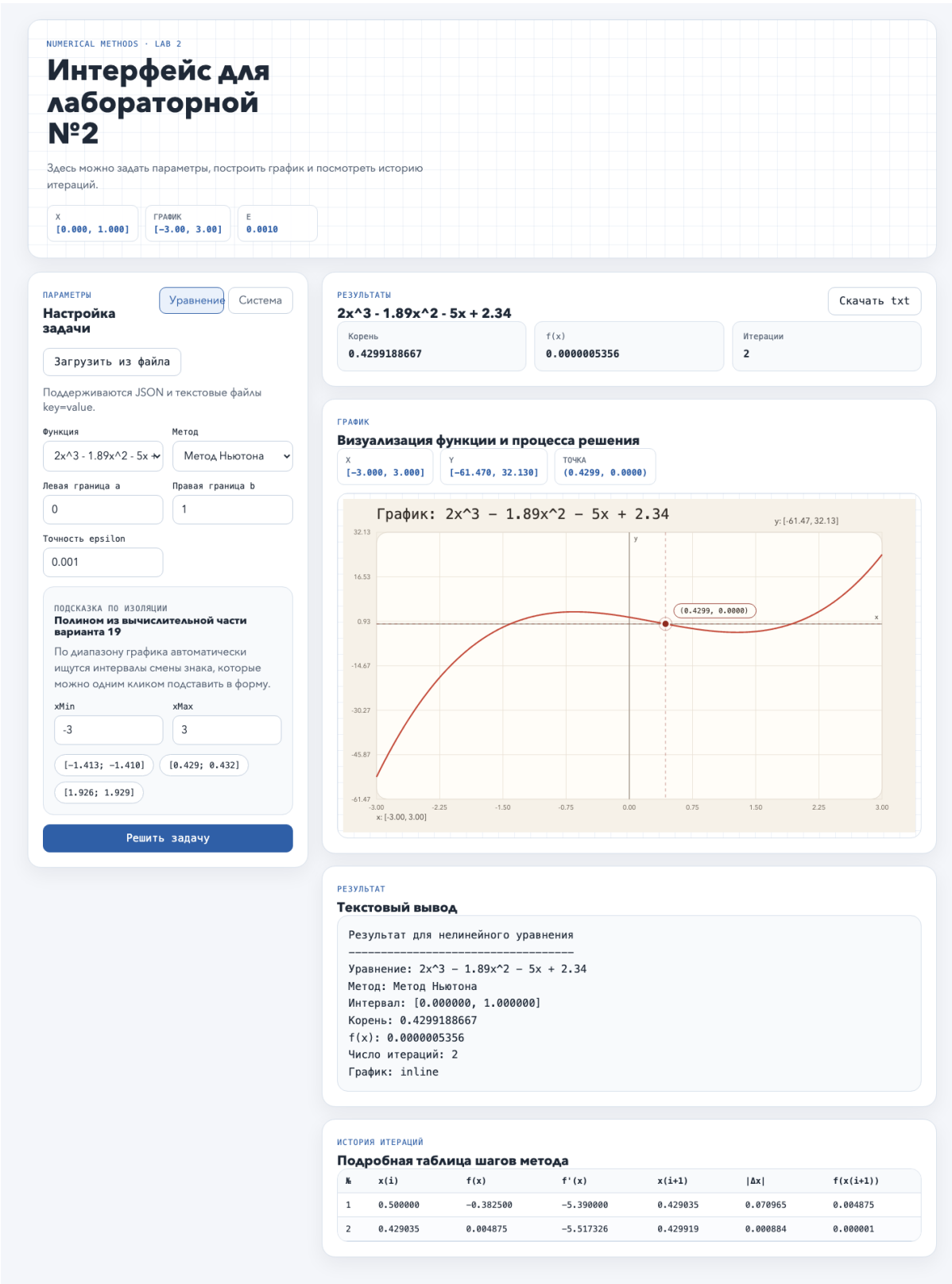
Для проверки работы программы подготовлены примеры запуска:

- `npm run demo:equation` — решение уравнения варианта 19 методом Ньютона;

- `npm run demo:system` — решение системы варианта 19 методом Ньютона;
- `npm run gui` — запуск графического интерфейса.

2.3 Скриншоты работы web-интерфейса

На рисунках ниже показана работа графического интерфейса: ввод параметров, построение графика, найденное решение и таблица итераций.



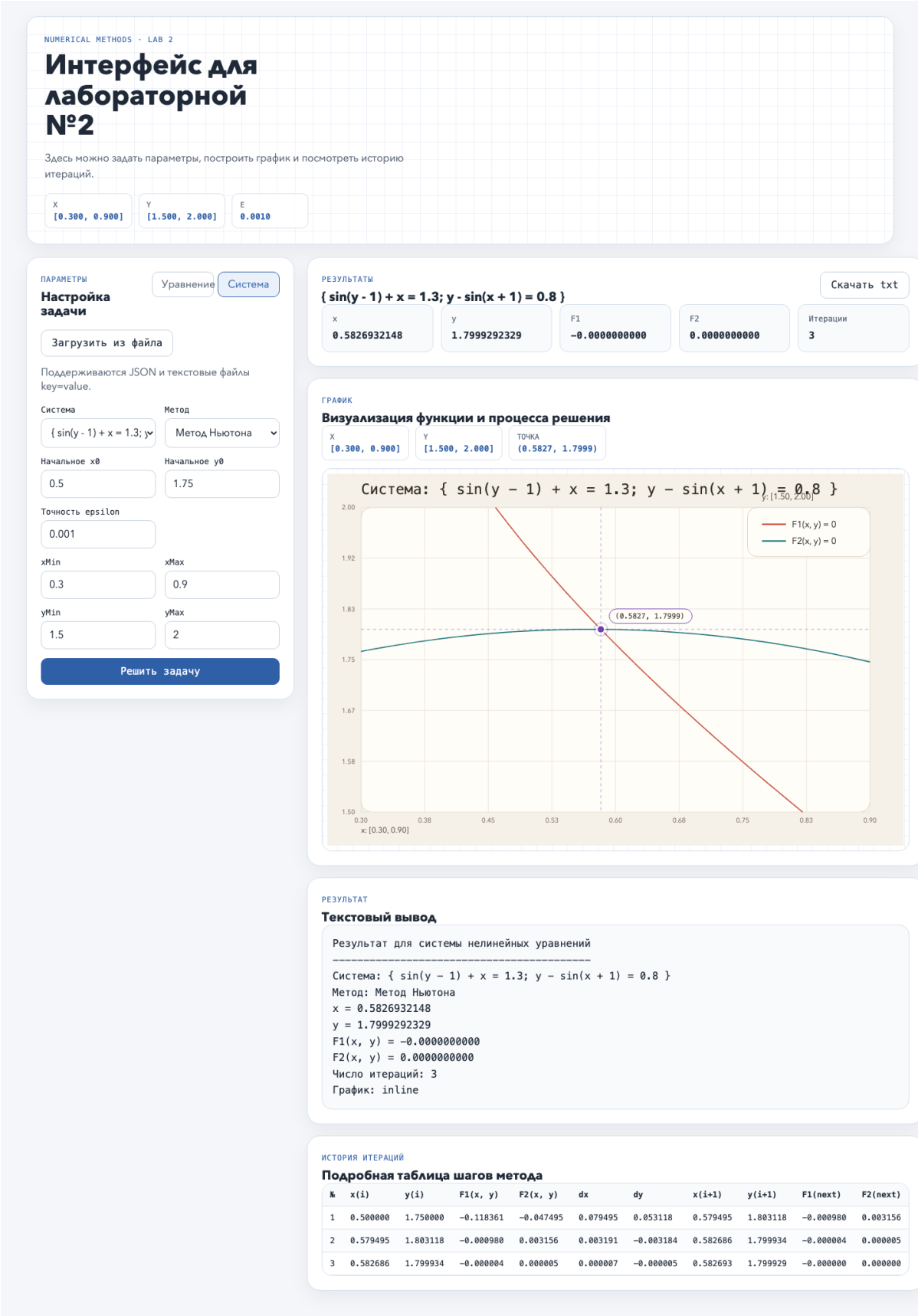


Рис. 8: Решение системы нелинейных уравнений в web-интерфейсе

3 Вывод

В работе выполнено графическое отделение корней уравнения варианта 19, выбраны интервалы изоляции и получены приближённые значения корней с точностью $\varepsilon = 10^{-2}$. Для системы варианта 19 найдено решение $(x^*, y^*) \approx (0.583; 1.800)$, невязки в найденной точке близки к нулю.

По таблице задания для варианта 19 в программе реализованы методы 2, 3, 5 и 6: хорд, Ньютона, простой итерации для уравнений и Ньютона для систем. В web-запуске метод Ньютона для уравнения сошёлся за 2 итерации, метод Ньютона для системы — за 3 итерации. Метод хорд не требует производной, но зависит от выбора интервала и фиксированного конца. Метод простой итерации применим после проверки условия $q < 1$; при нарушении этого условия программа выводит сообщение об ошибке. Web-интерфейс строит график, показывает таблицу итераций и сохраняет отображение графика при некорректном интервале.